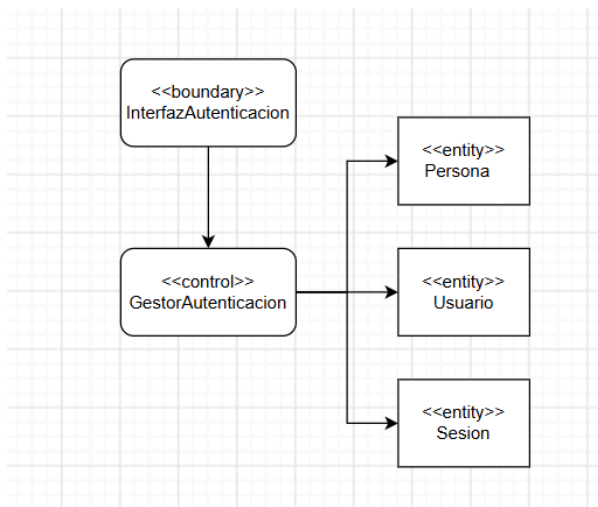


- Análisis y Diseño Orientado a Objetos:

➤ ¿Cuál es la Diferencia entre el Diagrama de Clases de Análisis y el Diagrama de Clases de Diseño? Realizar el Diagrama de Clases de Análisis del Ejercicio “a”.



. Diagrama de Clases de Análisis (Modelo del Dominio / Conceptual)

Representa los objetos de la realidad o las reglas de negocio sin preocuparse por la infraestructura. Omite detalles técnicos como métodos de acceso (getters/setters), librerías, claves primarias/foráneas o estructuras de datos complejas. Sirve para asegurar que los requisitos funcionales y el vocabulario del cliente estén bien entendidos.

. Diagrama de Clases de Diseño (Modelo de Software)

Traduce las clases conceptuales a componentes de software reales.

Incorpora la arquitectura del sistema: manejo de la interfaz de usuario, persistencia en base de datos, clases auxiliares, interfaces y frameworks.

Muestra navegación direccional entre clases, multiplicidades exactas, visibilidad de atributos/métodos y firma completa de operaciones.

- Patrones: Patrones GRASP.

➤ Realizar una explicación de cada uno de los Patrones GRASP. Además, investiga sobre el Patrón Modelo Vista Controlador (MVC). Explicar la vinculación que existe entre los patrones GRASP y el patrón MVC.

GRASP (General Responsibility Assignment Software Patterns)

Los patrones GRASP, formulados por Craig Larman, son principios o guías mentales para asignar responsabilidades a las clases y objetos en el diseño orientado a objetos.

1. Expert

- Concepto: Asigna una responsabilidad a la clase que posee la información necesaria para cumplirla.
- Utilidad: Mantiene la cohesión y reduce el acoplamiento al evitar que una clase externa "busque" datos en múltiples entidades para operar.

2. Creator

- Concepto: Define qué clase debe ser responsable de crear una nueva instancia de otra. Una clase A debe crear a B si A agrega, contiene, registra o usa de forma muy cercana a B.

- b. Utilidad: Promueve la encapsulación y reduce dependencias innecesarias al delegar la instanciación al contenedor lógico.
- 3. Controller
 - a. Concepto: Es el primer objeto más allá de la capa de interfaz de usuario que recibe y coordina la ejecución de una operación del sistema.
 - b. Utilidad: Separa la interfaz de usuario de la lógica del dominio, evitando que los eventos de la vista contengan lógica de negocio.
- 4. Low Coupling
 - a. Concepto: Mantiene las dependencias entre clases al mínimo.
 - b. Utilidad: Facilita la reutilización, reduce el impacto de los cambios en el código y simplifica el mantenimiento.
- 5. High Cohesion
 - a. Concepto: Asegura que los elementos de una clase estén fuertemente relacionados y enfocados en un propósito bien definido.
 - b. Utilidad: Evita clases multipropósito, manteniendo el código comprensible y testeable.
- 6. Polymorphism
 - a. Concepto: Maneja alternativas o variaciones de comportamiento basándose en el tipo de objeto mediante interfaces o herencia, en lugar de usar condicionales implícitos (if/switch).
 - b. Utilidad: Permite extender el software agregando nuevas clases sin modificar el código existente.
- 7. Pure Fabrication
 - a. Concepto: Crea una clase que no representa un concepto del dominio del negocio (ej. un servicio, repositorio o helper) para mantener un bajo acoplamiento y alta cohesión.
 - b. Utilidad: Evita sobrecargar a las clases del dominio con responsabilidades técnicas como persistencia, autenticación o registro de logs.
- 8. Indirection
 - a. Concepto: Introduce un componente intermedio (interfaces, adaptadores, controladores) para mediar entre dos elementos.
 - b. Utilidad: Desacopla dos componentes para que puedan cambiar de forma independiente.
- 9. Protected Variations
 - a. Concepto: Protege a los elementos del sistema frente a variaciones o cambios en otros componentes mediante la definición de interfaces o abstracciones estables.
 - b. Utilidad: Aísla los puntos de inestabilidad o cambio del resto de la aplicación.

Patrón de Arquitectura Modelo-Vista-Controlador (MVC)

El patrón MVC es un patrón de arquitectura de software que divide una aplicación en tres componentes interconectados para separar la presentación de la lógica:

- **Modelo:** Contiene la lógica del negocio, las reglas del sistema, el estado y las operaciones con los datos. Es independiente de la interfaz de usuario.
- **Vista:** Representa la interfaz gráfica y la presentación visual de los datos al usuario. Renderiza la información provista por el Modelo.
- **Controlador:** Actúa como intermediario. Capta las peticiones del usuario desde la Vista, invoca la lógica de negocio en el Modelo y determina qué Vista responder al usuario.

Vinculación entre Patrones GRASP y el Patrón MVC

MVC es un patrón arquitectónico de alto nivel, mientras que GRASP proporciona los principios de bajo nivel necesarios para estructurar internamente dicho patrón:

- **GRASP Controller vs. MVC Controller:** El principio GRASP *Controller* es el fundamento directo de la capa *Controller* en MVC. Define formalmente que las solicitudes de la vista deben ser recibidas por un objeto desacoplado de la interfaz.
- **High Cohesion y Low Coupling en la separación MVC:** La división entre Modelo, Vista y Controlador es una aplicación directa de *Bajo Acoplamiento y Alta Cohesión*. Evita que la interfaz (Vista) modifique bases de datos directamente o que el Modelo conozca botones y pantallas.
- **Pure Fabrication en el Modelo:** Las clases de lógica de negocio o servicios que conforman el *Modelo* en una arquitectura MVC (como servicios REST, DAOs o Repositorios) suelen ser *Invencciones Puras* creadas para no colapsar las entidades del dominio con código técnico.
- **Protected Variations e Indirection entre capas:** El *Controlador* en MVC actúa como un punto de *Indirección y Variación Protegida*. Si la interfaz gráfica (Vista) cambia completamente (ej. de web a móvil), el Modelo no se ve afectado gracias a la capa intermedia que amortigua el cambio.

➤ Explicar los conceptos de FrontEnd y BackEnd. Explicar la arquitectura en Capas y como esta se vincula con el patrón MVC.

FrontEnd y BackEnd

En el desarrollo de software, estos términos dividen una aplicación según la ubicación donde se ejecuta el código y su responsabilidad funcional:

- **FrontEnd (Lado del Cliente):** Es la capa visible e interactiva de la aplicación. Corresponde a todo lo que se ejecuta directamente en el dispositivo del usuario (navegador web, aplicación móvil o de escritorio). Su objetivo es presentar datos, capturar eventos de entrada y brindar una experiencia de usuario fluida (usando tecnologías como HTML, CSS, JavaScript, React o Angular).
- **BackEnd (Lado del Servidor):** Es la capa lógica e invisible para el usuario. Se ejecuta en servidores (locales o en la nube) y gestiona el procesamiento pesado: reglas de negocio, autenticación, seguridad, integración con APIs externas y persistencia en bases de datos (utilizando lenguajes como Java, Python, C#, Node.js o Go).

Arquitectura en Capas

Es un patrón de diseño arquitectónico que organiza el código en secciones horizontales independientes, donde **cada capa tiene una responsabilidad única y sólo interactúa de forma directa con sus capas adyacentes.**

Las capas estándar son:

1. **Capa de Presentación:** Gestiona la interfaz gráfica y la interacción directa con el cliente.
2. **Capa de Servicios:** Contiene las reglas, validaciones, algoritmos y flujos lógicos de la aplicación.
3. **Capa de Persistencia / Acceso a Datos (DAL/Repositories):** Se encarga exclusivamente de las consultas y transacciones con el almacenamiento físico (bases de datos, archivos, etc.).

Vinculación entre Arquitectura en Capas y el Patrón MVC

Existe una relación directa y complementaria entre ambos conceptos, aunque operan a distintas escalas del sistema: **MVC se aplica a la capa de presentación/aplicación,** mientras que la **Arquitectura en Capas organiza todo el sistema de software.**

Componente MVC	Capa Arquitectónica Correspondiente	Cómo se vinculan

Vista (View)	Capa de Presentación (FrontEnd)	Renderiza la interfaz y muestra la información al usuario.
Controlador (Controller)	Límite entre Presentación y Negocio	Recibe las solicitudes HTTP/eventos de la Vista y delega el trabajo real a la Capa de Negocio.
Modelo (Model)	Capa de Negocio + Capa de Persistencia	Contiene las entidades del dominio, las reglas del sistema y las estructuras de datos que se recuperan del BackEnd.

- Spring: Framework Spring Boot.

➤ ¿Qué es Spring Boot? ¿Cuál es su Sitio Web Oficial? ¿Cómo se accede a la documentación?

Spring Boot es un marco de trabajo (*framework*) de código abierto basado en Java que simplifica el desarrollo de aplicaciones web y microservicios. Su objetivo principal es acelerar la configuración inicial eliminando la necesidad de definir archivos XML de configuración complejos o desplegar servidores web manualmente, aplicando el principio de "convención sobre configuración".

Incluye un servidor web embebido (como Tomcat), gestores de dependencias simplificados (*starters*) y autoconfiguración inteligente que permite tener una aplicación ejecutable en minutos.

Sitio Web Oficial y Documentación

- Sitio Web Oficial: spring.io/projects/spring-boot
- Acceso a la Documentación:
 1. Se accede directamente ingresando a la sección de documentación oficial en docs.spring.io/spring-boot.

➤ ¿Qué es Spring Tools? ¿Cuál es su Sitio Web Oficial?

Spring Tools (oficialmente conocido como **Spring Tools 4**) es el conjunto de plugins y herramientas de desarrollo oficial mantenido por el equipo de Spring para facilitar la

creación de aplicaciones con Spring y Spring Boot. Proporciona soporte avanzado para autocompletado de código, navegación inteligente de dependencias, detección de errores en tiempo de compilación y herramientas de depuración (*debugging*) específicas para el ecosistema Spring.

Está disponible principalmente en dos formatos:

- Como una distribución completa del IDE basada en Eclipse (**Spring Tool Suite 4 / STS4**).
- Como extensiones para IDEs y editores populares como **Visual Studio Code** y **Eclipse**.

Sitio Web Oficial

- URL Directa: spring.io/tools

➤ ¿Qué es Spring Initializr? ¿Cuál es su Sitio Web Oficial?

Spring Initializr es una herramienta web oficial del ecosistema Spring que funciona como un generador de plantillas y estructuras de proyectos. Permite inicializar rápidamente una aplicación Spring Boot seleccionando la versión del lenguaje (Java, Kotlin o Groovy), el gestor de dependencias (Maven o Gradle), la versión de Spring Boot y agregando los módulos o dependencias necesarios (como Spring Web, Spring Data JPA, Spring Security, MariaDB Driver, etc.).

Al finalizar la selección, la herramienta genera un archivo comprimido **.zip** con la estructura base completa del proyecto lista para importar y ejecutar en cualquier IDE.

Sitio Web Oficial

- URL Directa: start.spring.io

➤ ¿Cuáles son las anotaciones en Spring Boot con las que se identifican una clase de tipo controlador, una clase de tipo servicio, una clase de tipo repositorio? Explicar que implicancia tiene cada una, además de explicar cuál es el significado de una clase Controlador, una Clase de Servicio y una clase de Acceso a Datos.

Las anotaciones de estereotipo en Spring Boot le indican al contenedor de Inversión de Control (IoC) que instancie y gestione esas clases como componentes reutilizables (*Beans*).

Capa	Anotación	Significado de la Clase	Implicancia de la Anotación
Controlador	<code>@RestController</code> (o <code>@Controller</code>)	Maneja las peticiones HTTP entrantes, parsea la entrada del cliente, invoca la lógica de negocio y responde con datos (JSON/XML) o vistas HTML.	Registra la clase como un componente web en el contexto de Spring. En el caso de <code>@RestController</code> , combina <code>@Controller</code> y <code>@ResponseBody</code> , convirtiendo automáticamente los retornos de los métodos a JSON/XML.
Servicio	<code>@Service</code>	Concentra la lógica de negocio de la aplicación, coordina los flujos de trabajo de los datos, aplica validaciones complejas y maneja transacciones de negocio.	Especialización de <code>@Component</code> . Le indica a Spring que esta clase contiene la capa de servicio y permite aplicar aspectos globales como la gestión transaccional con <code>@Transactional</code> .
Repositorio	<code>@Repository</code>	Se encarga de la persistencia y acceso a datos . Ejecuta operaciones CRUD y consultas directamente sobre la base de datos o almacenamiento físico.	Especialización de <code>@Component</code> para la capa de persistencia. Además de marcar la clase, activa la traducción automática de excepciones de la base de datos a excepciones de Spring (<code>DataAccessException</code>).

Comportamiento en Conjunto

Gracias a estas anotaciones, Spring aplica **Inyección de Dependencias** mediante anotaciones como **@Autowired** (o constructores).

El flujo de trabajo habitual funciona así:

1. El cliente envía un *request* HTTP al **@RestController**.
2. El **@RestController** delega la lógica al **@Service**.
3. El **@Service** llama al **@Repository** para consultar o persistir los datos.
4. El **@Repository** interactúa con la base de datos y retorna los resultados hacia arriba.

➤ Explicar la Diferencia entre el Patrón DAO y Repository de Spring Boot.

Criterio	Patrón DAO	Patrón Repository (Spring Boot)
Nivel de Abstracción	Bajo nivel (cercano a la base de datos / tabla).	Alto nivel (cercano al dominio del negocio / entidades).
Punto de Partida	Las tablas y operaciones SQL.	Las entidades y conceptos del dominio.
Implementación en Spring	Requiere escribir clases que implementen la interfaz (con EntityManager o JdbcTemplate).	Spring genera la implementación automáticamente al extender de JpaRepository .
Relación con las Clases	Suele haber un DAO por cada tabla de la base de datos.	Hay un Repository por cada Agregado Raíz del dominio.

Manejo de Consultas	Consultas SQL o HQL explícitas dentro de los métodos.	Consultas derivadas por nombre de método (<i>Query Methods</i>) o <i>@Query</i> .
----------------------------	---	---

➤ ¿Para qué sirve el archivo pom.xml en un proyecto Spring Boot, que formato tiene y cual puede ser su contenido?

El archivo **pom.xml** (*Project Object Model*) es el corazón de la configuración de un proyecto gerenciado por Apache Maven en Spring Boot.

¿Para qué sirve?

- **Gestión de dependencias:** Descarga, actualiza y gestiona automáticamente las librerías externas que necesita el proyecto (ej. Spring Web, drivers de base de datos, Lombok), resolviendo las dependencias transitivas (librerías que requieren otras librerías).
- **Gestión de la construcción (*Build*):** Define cómo se compila el proyecto, cómo se ejecutan las pruebas unitarias y cómo se empaqueta la aplicación en un archivo ejecutable (*.jar* o *.war*).
- **Definición de metadatos:** Especifica el nombre, versión, descripción del proyecto y la versión de Java utilizada.
- **Control de plugins:** Configura herramientas adicionales para el proceso de compilación, como el plugin de Spring Boot para empaquetado.

¿Qué formato tiene?

Tiene formato **XML** (*Extensible Markup Language*). Utiliza una estructura jerárquica de etiquetas (*tags*) anidadas definida por el esquema oficial de Maven.

➤ Explicar el Patrón Inyección de Dependencia. ¿Cómo se aplica el patrón en Spring Boot?

Inyección de Dependencias (Dependency Injection - DI)

Es un patrón de diseño orientado a objetos que implementa el principio de **Inversión de Control (IoC)**. Consiste en transferir la responsabilidad de crear y suministrar los objetos que una clase necesita (*sus dependencias*) a un contenedor externo, en lugar de que la propia clase los instancie directamente con el operador **new**.

¿Cómo se aplica en Spring Boot?

Spring Boot actúa como el Contenedor IoC (a través de su `ApplicationContext`). Spring escanea las clases marcadas con anotaciones de componente (`@Component`, `@Service`, `@Repository`, `@RestController`), las instancia, gestiona su ciclo de vida y las almacena en memoria como Beans. Luego, inyecta esos Beans automáticamente donde sean requeridos.

- Diagrama de Secuencia:

➤ ¿Qué es el Diagrama de Secuencia y para qué sirve? Describir los elementos gráficos que utiliza el diagrama de Secuencia. ¿Cuál es la diferencia entre el Diagrama de Secuencia de Análisis y el Diagrama de Secuencia de Diseño? Explicar un ejemplo de ABM de una entidad Nacionalidad que posee el atributo nombre, cómo se representan en un Diagrama de Secuencia de Diseño y como son los mensajes entre la clase Controladora, la clase de Servicio y la clase de Acceso a Datos.

¿Qué es el Diagrama de Secuencia y para qué sirve?

El Diagrama de Secuencia es un diagrama de comportamiento dinámico dentro del estándar UML. Muestra cómo interactúan los objetos entre sí a lo largo del tiempo para llevar a cabo un caso de uso o escenario particular.

Sirve para:

- Visualizar el flujo de control, la lógica y el intercambio de mensajes paso a paso en el tiempo.
- Validar que la arquitectura del sistema soporta correctamente los requerimientos funcionales.
- Identificar claramente las responsabilidades, métodos y firmas que debe tener cada clase en la implementación.

Elementos Gráficos del Diagrama de Secuencia

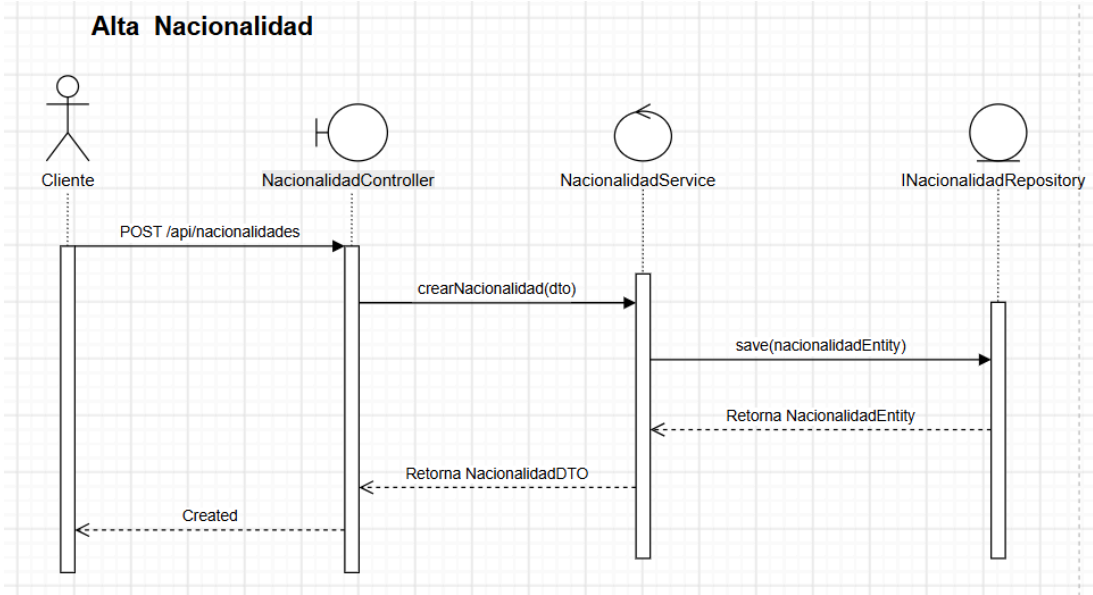
- **Objetos / Participantes:** Rectángulos en la parte superior que representan las instancias de las clases o actores.
- **Línea de Vida (*Lifeline*):** Línea vertical de trazo discontinuo que desciende desde el participante y representa su existencia en el tiempo.
- **Barra de Activación (Foco de Control):** Rectángulo vertical delgado sobre la línea de vida que indica el periodo durante el cual el objeto está ejecutando una operación.
- **Mensaje Sincrónico:** Flecha con punta maciza que indica una llamada a un método. El emisor se bloquea a la espera de la respuesta.

- **Mensaje Asincrónico:** Flecha con punta abierta que envía una señal sin esperar respuesta para continuar.
- **Mensaje de Retorno:** Flecha discontinua que representa la devolución de datos o control desde el método invocado.
- **Fragmentos Combinados (Lógica de control):** Marcos que encierran interacciones para modelar lógica condicional o repetitiva:
 - **alt:** Alternativa condicionada (**if - else**).
 - **opt:** Opción condicional (**if** simple).
 - **loop:** Bucle o repetición (**for / while**).

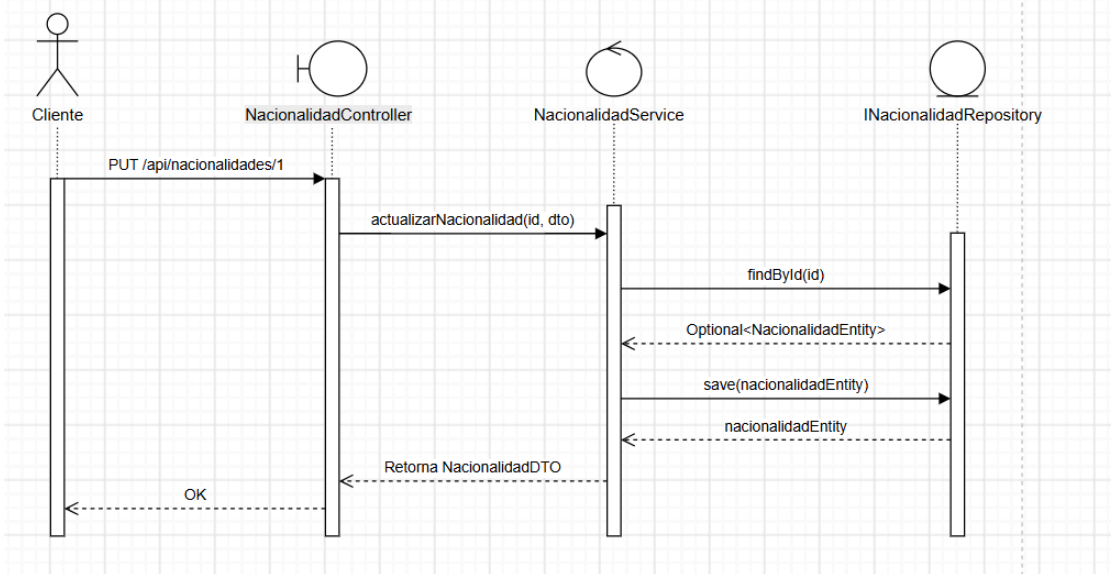
Diferencia: Secuencia de Análisis vs. Secuencia de Diseño

Criterio	Diagrama de Secuencia de Análisis	Diagrama de Secuencia de Diseño
Enfoque	Mostrar <i>qué</i> interacciones ocurren desde el punto de vista funcional/negocio.	Mostrar <i>cómo</i> se ejecuta técnicamente en código.
Participantes	Actores del usuario y estereotipos de análisis (<<boundary>> , <<control>> , <<entity>>).	Clases técnicas reales (Controller , Service , Repository , DTO , Entity).
Mensajes	Nombres descriptivos o conceptuales de casos de uso (ej. <i>crearNacionalidad</i>).	Firmas de métodos precisas de lenguaje de programación (ej. save(nacionalidad)).
Detalle Técnico	Omite tipos de retorno, respuestas HTTP, DTOs y excepciones.	Muestra retornos explícitos, respuestas en objetos

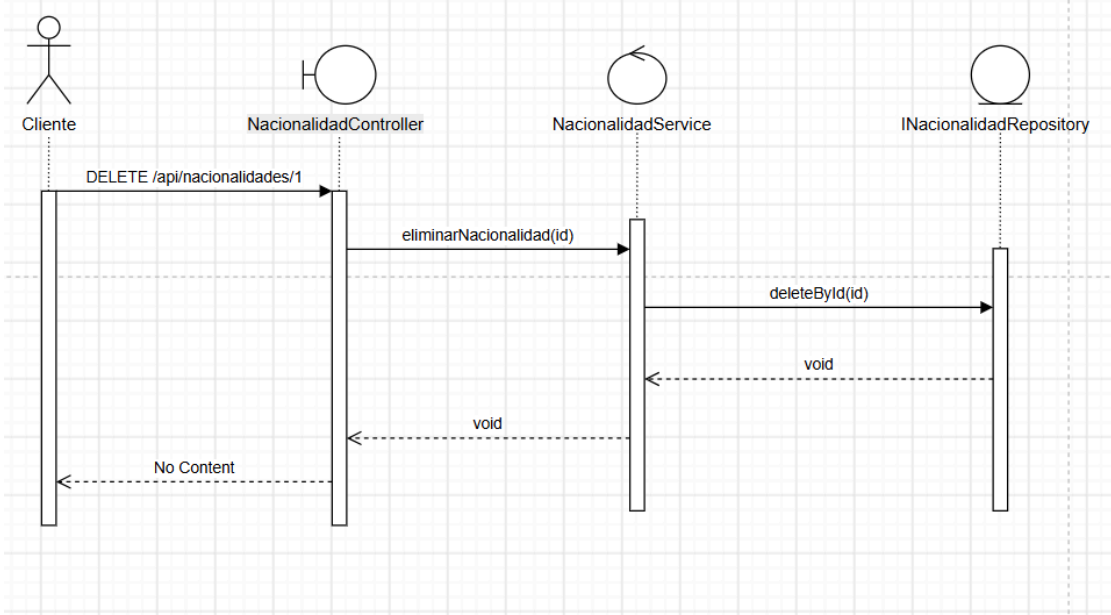
		JSON/HTTP y parámetros de métodos.
--	--	------------------------------------



Modificación Nacionalidad



Baja Nacionalidad



- Diagrama de Paquetes:

➤ ¿Qué es el Diagrama de Paquetes y para qué sirve? Describir los elementos gráficos que utiliza el diagrama de Paquetes.

¿Qué es el Diagrama de Paquetes y para qué sirve?

El **Diagrama de Paquetes** es un diagrama de estructura dentro del estándar UML que organiza los elementos del sistema (clases, interfaces, caseras de uso u otros diagramas) en **agrupaciones lógicas llamadas paquetes**.

Sirve para:

- **Gestionar la complejidad:** Organiza sistemas grandes dividiéndolos en módulos o subsistemas manejables.
- **Modelar la arquitectura:** Muestra claramente las capas del sistema (ej. Presentación, Negocio, Datos) y cómo se relacionan entre sí.
- **Controlar el acoplamiento:** Permite visualizar y minimizar las dependencias directas entre diferentes partes del código.

Elementos Gráficos del Diagrama de Paquetes

- **Paquete (*Package*):** Se representa como una carpeta de archivos (un rectángulo grande con una pestaña pequeña adjunta en la esquina superior izquierda). Contiene el nombre del paquete en el centro o dentro de la pestaña.
- **Dependencia (`<<use>>` / `<<import>>`):** Se dibuja como una **línea discontinua con flecha abierta**. Apunta desde el paquete dependiente (el que necesita algo) hacia el paquete del cual depende.
- **Paquetes Anidados:** Un paquete dibujado físicamente dentro de otro rectángulo de paquete para indicar jerarquía de subpaquetes (ej. [com.ejemplo.proyecto.controllers](#)).
- **Estereotipos:** Etiquetas de texto encerradas entre comillas angulares (`<< >>`) que especifican el rol técnico o arquitectónico del paquete:
 - `<<layer>>`: Para capas de la arquitectura (ej. `<<layer>> Presentacion`).
 - `<<subsystem>>`: Para módulos independientes del sistema.
 - `<<import>>` / `<<access>>`: Indican el tipo de visibilidad o relación de importación entre paquetes.
- **Visibilidad de Elementos:** Prefijos opcionales que indican la accesibilidad de los componentes dentro del paquete (+ Público, - Privado, # Protegido).

- *FrontEnd: Thymeleaf*

➤ Identificar las principales características de Thymeleaf

Thymeleaf es un motor de plantillas Java moderno diseñado para procesar y renderizar HTML, XML, JavaScript, CSS y texto plano, altamente utilizado en el ecosistema Spring Boot.

- **HTML 5 Nativo (*Natural Templates*):** Permite diseñar plantillas que son HTML válido. Se pueden abrir directamente en cualquier navegador web sin necesidad de ejecutar un servidor backend, lo que facilita el trabajo colaborativo con diseñadores maquettadores.
- **Integración Total con Spring Boot / Spring MVC:** Se integra de forma transparente con el manejo de formularios, validaciones, binding de objetos del modelo, mensajería i18n (internacionalización) y Spring Security.
- **Procesamiento en el Lado del Servidor (SSR):** Evalúa expresiones e inyecta los datos dinámicos provistos por los controladores en el servidor antes de enviar el HTML final parseado al cliente.

➤ ¿Cuál es su sitio web Oficial? ¿Cómo se accede a la documentación oficial?

Sitio Web Oficial, URL Directa: thymeleaf.org

Acceso a la Documentación Oficial: Se accede ingresando directamente a la sección de documentación en thymeleaf.org/documentation.html.

➤ ¿Cómo se vincula Thymeleaf con HTML?

Thymeleaf se vincula con HTML mediante la declaración de su espacio de nombres (namespace) XML en la etiqueta raíz `<html>` del documento. Esto permite que el navegador reconozca y valide las etiquetas y atributos especiales que empiezan con el prefijo `th:`

Sintaxis de Vinculación: Se agrega el atributo `xmlns:th` apuntando a la URI oficial de Thymeleaf

Puntos Clave de la Vinculación

- **Namespace `xmlns:th`:** Le indica al motor de plantillas en el servidor qué atributos debe interceptar, evaluar y procesar antes de enviar el HTML final al cliente.
- **Compatibilidad con HTML estático:** Si abres el archivo `.html` directamente en el navegador desde el disco local (sin pasar por Spring Boot), el navegador ignora el namespace `xmlns:th` y muestra el contenido HTML estándar que está dentro de la etiqueta (`Texto por defecto`), permitiendo diseñar la maquetación sin levantar el servidor.